TDT4265 Mini-Project

Snow Pole Detection for Autonomous Driving

Gustavo Garzo Chumilla Jordan Ikukele Bomolo Mia

NTNU – Computer Vision and Deep Learning

April 2026

Outline

Presentation roadmap

1. Background & Motivation
2. Approach / Strategy
3. Data Analysis
4. Methods & Models
5. Results
6. Discussion
7. Key Learning Points
8. Sustainability

Background & Motivation

- Autonomous driving systems perform well in ideal road conditions, but snowy roads remain challenging because boundaries and lane markings become hard to see.
- In winter, **snow poles** provide a robust visual cue for road localisation because they remain visible even when the road surface is covered.
- Our project therefore focuses on **single-class object detection** of snow poles in realistic winter driving scenes from Trondheim.

Main challenge: poles are thin, visually small, and often far from the camera.

The final goal is not only accuracy, but also a detector that could realistically support **real-time edge deployment**.

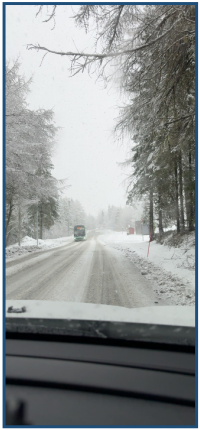
Problem framing

- task: one-class object detection
- application: winter road localisation
- constraint: edge-ready, real-time inference
- evaluation: Precision, Recall, mAP@50 and mAP@0.5:0.95

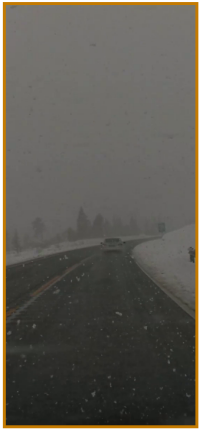
Background & Motivation – Visual Examples

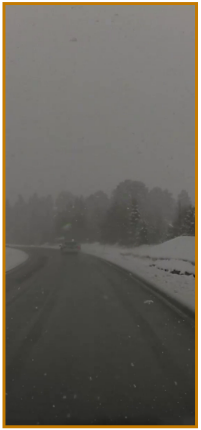
Representative training images

Road_poles_iPhone



roadpoles_v1





Approach / Strategy

Project strategy

- We first treated the task as a **dataset understanding problem**, not only a model training problem.
- We analysed **both available leaderboards separately**: `Road_poles_iPhone` and `roadpoles_v1`.
- The goal of the EDA was to understand whether both datasets had similar object scale, geometry and scene structure.

Why this matters

The course brief explicitly asks for model development to be guided by exploratory data analysis, so this step is part of the methodology rather than a side note.

Pipeline

Problem framing

↓

EDA on both datasets

↓

Resolution and model selection

↓

Training, validation and leaderboard submission

Approach / Strategy – Key Reasoning

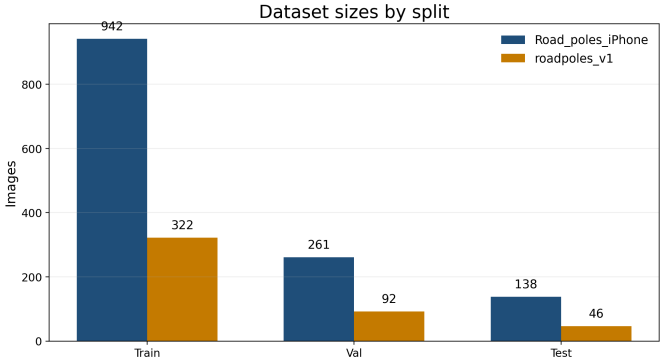
What the EDA needed to answer

- Are the two datasets similar enough that we should interpret them as the same task?
- Is the main difficulty object density, object scale, object geometry, or scene structure?
- Which data properties are most likely to explain differences in final leaderboard performance?

How the analysis informs the rest of the presentation

- If poles are very small, input resolution becomes a critical design choice.
- If one dataset is geometrically harder, lower results there do not necessarily mean the model is worse overall.
- This gives context for the model and training decisions shown in the next section.

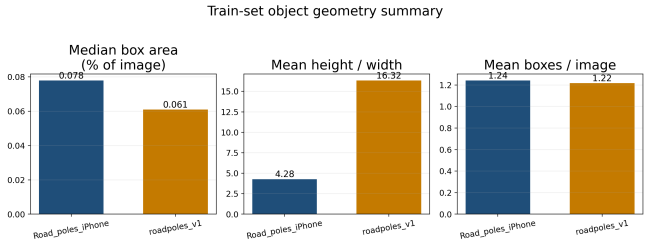
Data Analysis – Dataset Overview



Key observations

- `Road_poles_iPhone` is the larger dataset:
  - 942 train
  - 261 val
  - 138 test
- `roadpoles_v1` is smaller:
  - 322 train
  - 92 val
  - 46 test
- The test labels are hidden in both datasets, so the EDA must rely on train and validation splits only.

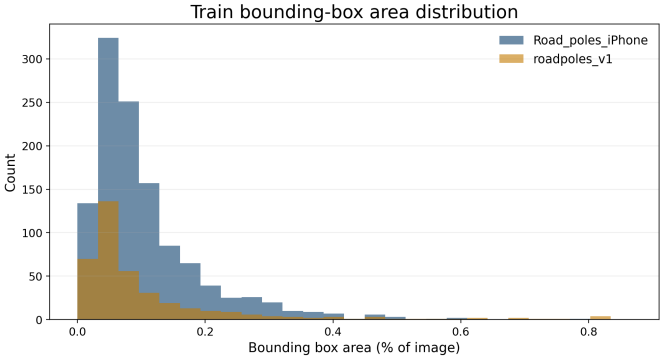
Data Analysis – Object Geometry



Why this is relevant

- `roadpoles_v1` is the **harder benchmark geometrically**:
  - smaller median box area
  - much larger height/width ratio
- Both datasets are sparse at around **1.2 poles per image**.
- The difficulty is therefore not crowded scenes, but detecting small and thin roadside objects.

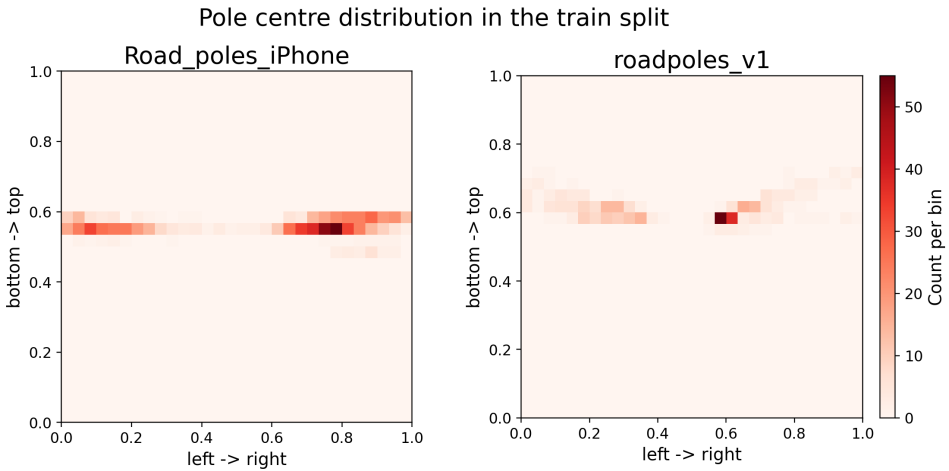
Data Analysis – Object Scale



Interpretation

- In both datasets, poles occupy only a **tiny percentage of the image area**.
- This confirms that the task is dominated by **small-object detection**.
- It also helps explain why image resolution is an important variable in the methods section.

Data Analysis – Spatial Distribution



Pole centres cluster in a narrow horizontal band in both datasets, consistent with road-side object placement.

Data Analysis – Spatial Distribution Insights

Interpretation

- The two datasets are not random image collections, but structured driving scenes with a clear spatial bias.
- This matters when interpreting results: errors are more likely to come from **small-object visibility and localisation** than from heavy clutter.
- These observations directly motivate the model and training choices shown in the next section.

Overview – Two Architectures

Model 1 – YOLOv8n Baseline

- Single-stage detector
- You Only Look Once – one forward pass
- Lightweight baseline for fast iteration
- Separate notebooks prepared for both datasets

Model 2 – Faster R-CNN

- Two-stage detector
- Region Proposal Network + classification
- Torchvision pretrained backbones + FPN
- Strongest validated results on both datasets

Both model families were used on both leaderboards: `Road_poles_iPhone` and `roadpoles_v1`

Model 1 – YOLO Baseline Architecture

Why YOLO as a baseline?

- Single-stage detector with very fast training and inference
- Good baseline for checking whether the task can be solved with a lightweight model
- Easy to train and export with the Ultralytics workflow
- Relevant to the deployment goal because it is edge-friendly

Notebook setup used in the repo:

- `yolov8n.pt` pretrained checkpoint
- 100 epochs
- `imgsz=640`
- Separate notebooks prepared for each dataset

Model stats

- Variant: YOLOv8n
- Input size: 640×640
- Classes: 1 (pole)
- Pretrained: COCO
- Device: CUDA in notebook config
- Role in project: baseline detector

Model 1 – YOLO Training Config

Training code

```
model = YOLO("yolov8n.pt")
model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    device=0
)
```

- The YOLO notebooks use the same simple training call for quick baseline experiments
- Fine-tuned from COCO pretrained weights
- Used mainly as the fast reference model for comparison

Model 2 – Faster R-CNN Architecture

Two-stage detection pipeline:

1. **Stage 1 – RPN:** Scans feature map, proposes candidate regions containing objects
2. **Stage 2 – ROI Head:** Classifies each proposal and refines the bounding box

FPN backbone: Extracts features at multiple scales simultaneously – important for poles at different distances **Key difference from YOLO:**

Two passes = slower, but better suited to careful localisation of thin objects

Architecture

- Backbone family: pretrained Faster R-CNN + FPN
- Pretrained: COCO
- Head: replaced for 2 classes (background + pole)
- Optimiser: AdamW
- Scheduler: CosineAnnealingLR
- Epochs: 100

Model 2 – Faster R-CNN Training Config

Custom dataset class – YOLO labels to Faster R-CNN format

```
# Convert normalised YOLO format to absolute pixel coords
x1 = (cx - bw/2) * W
y1 = (cy - bh/2) * H
x2 = (cx + bw/2) * W
y2 = (cy + bh/2) * H
# Labels: 0 = background, 1 = pole
```

Augmentations (train only)

```
train_transform = T.Compose([
    T.ToTensor(),
    T.ColorJitter(brightness=0.4, contrast=0.4,
                  saturation=0.3, hue=0.1),
    T.RandomHorizontalFlip(p=0.5),
])
```

Results – Validation Set Metrics

Dataset	Precision	Recall	mAP@50	mAP@0.5:0.95
Road_poles_iPhone	**0.7796**	**0.8097**	**0.9859**	**0.7796**
roadpoles_v1	0.6701	0.7124	0.9850	0.6701

Table: Faster R-CNN validation results on both datasets

- The strongest completed validation runs in the notebooks are the Faster R-CNN runs
- Performance is clearly higher on `Road_poles_iPhone` than on `roadpoles_v1`
- This matches the EDA, where `roadpoles_v1` appeared smaller, thinner, and harder

Results – Leaderboard

Road poles iPhone leaderboard

- Submitted Faster R-CNN predictions
- Score: $\approx$**69%**

roadpoles v1 leaderboard

- Submitted Faster R-CNN predictions
- Score: $\approx$**66%**

Note: Hidden test scores are lower than validation – expected due to unseen weather and lighting conditions

Results – Same Model, Two Datasets

Using the same model family on both datasets makes the dataset difficulty easier to interpret:

- **mAP@50:** almost identical on both datasets, so poles are found reliably in both cases
- **mAP@0.5:0.95:** noticeably higher on `Road_poles_iPhone`, which means tighter localisation there
- The stricter metric reveals the real difficulty difference better than mAP@50 alone

This supports the EDA conclusion that `roadpoles_v1` is the harder benchmark for thin-object localisation.

	@50	**@0.5:0.95**
Road_poles_iPhone	0.9859	0.7796
roadpoles_v1	0.9850	0.6701

Difference in mAP@50: 0.0009
Difference in mAP@0.5:0.95: 0.1095

Discussion – Why Faster R-CNN Performed Best

Three key factors

1. **Two-stage localisation:** proposing regions first and refining them afterwards seems to help on thin poles.
2. **Multi-scale features:** the FPN-based Faster R-CNN pipeline is well aligned with small and distant roadside objects.
3. **EDA consistency:** the same model performed better on the easier dataset and worse on the harder one, which makes the results more interpretable.

In our completed runs, Faster R-CNN was the strongest validated approach on both datasets, even though it is slower than YOLO.

Discussion – Limitations & Future Work

Current limitations:

- `roadpoles_v1` remains much harder than `Road_poles_iPhone`
- Hidden test gap still suggests domain shift (weather, lighting)
- Faster R-CNN is accurate, but slower for real-time edge deployment
- No pseudo-labelling of unlabelled RoadPoles-MSJ data

Future improvements:

- Run a cleaner hyperparameter sweep for both architectures
- Pseudo-label RoadPoles-MSJ with best YOLO model
- Try lighter Faster R-CNN backbones for faster inference
- Test time augmentation (TTA) for better test scores
- Ensemble YOLO + Faster R-CNN predictions

Key Learning Points

1 – EDA predicted the ranking between datasets

The EDA suggested that `roadpoles_v1` would be the harder benchmark because the poles are smaller and more elongated. The Faster R-CNN results later confirmed exactly that pattern.

2 – Understanding two-stage detection

Implementing Faster R-CNN manually revealed how the RPN and ROI pooling work together. The label format conversion from YOLO normalised boxes to absolute pixel coordinates highlighted how different frameworks expect data.

3 – Applied deep learning is mostly engineering

A significant portion of the project was path issues, yaml configuration, CUDA driver mismatches, and dataset format debugging. These are the real-world challenges in any CV deployment scenario.

Sustainability – Compute Energy

Hardware used:

- NVIDIA RTX 4090 (Cybele lab)
- TDP: 450W

Approximate training time:

- YOLOv8s (150 epochs, imgsz=1280): ≈2.5h
- Faster R-CNN (100 epochs): ≈1.5h
- Total: ≈**4 hours**

Energy consumed:

$$E = 0.45\,\text{kW} \times 4\,\text{h} = \mathbf{1.8}\,\text{kWh}$$

How long could we code?

MacBook Pro M3 consumption:
≈**45 W under load**

$$t = \frac{1800\,\text{Wh}}{45\,\text{W}} = \mathbf{40}\,\text{hours}$$

≈ a full work week of laptop use

Relatively low footprint – fine-tuning from pretrained weights avoided training from scratch

Summary

Dataset	mAP@50	mAP@0.5:0.95	Precision	Recall
Road_poles_iPhone	0.9859	**0.7796**	**0.7796**	**0.8097**
roadpoles_v1	0.9850	0.6701	0.6701	0.7124

- **Best validated model family:** Faster R-CNN in the completed notebook runs
- **Main dataset conclusion:** `Road_poles_iPhone` is easier than `roadpoles_v1`, consistent with the EDA
- **Energy cost:** 1.8 kWh total $\approx$ 12 km in a Tesla Model Y

Thank you for listening